CEE6501 — Lecture 11.2

Geometric Nonlinear Truss Analysis

Learning Objectives

By the end of this lecture, you will be able to:

- explain how the **local coordinate system** for a truss member changes in geometrically nonlinear analysis
- define the **corotational / Eulerian local coordinate system** for a plane truss member
- derive the **member axial force-displacement relation** in the local system
- compute the deformed member length and current direction cosines from the global nodal displacements
- transform the local axial force q into the global member end-force vector $\mathbf{f}$
- explain why the resulting member relations are nonlinear even when the material remains linearly elastic

Agenda

- Part 1 — What Changes in the Nonlinear Truss Setup?
- Part 2 — Member Force-Displacement Relations in the Local System
- Part 3 — Local to Global Transformations
- Part 4 — Example (Given Displacements)

Part 1 — What Changes in the Nonlinear Truss Setup?

What Stays the Same

Much of the overall structural setup remains the same as in the linear truss case.

We still need to:

- establish a **global coordinate system**
- number the **joints** and **members**
- identify the global displacement DOFs
- identify restrained and free coordinates

So the overall direct-stiffness workflow is familiar.

What changes is the way we define the **member local coordinate system** and the way we express the member force-displacement relations.

What Changes

In **linear truss analysis**, the member local coordinate system is fixed in the initial undeformed configuration.

In **geometrically nonlinear truss analysis**, it is more convenient to use a local coordinate system that:

- remains attached to the member
- translates with the member
- rotates with the member as the structure deforms

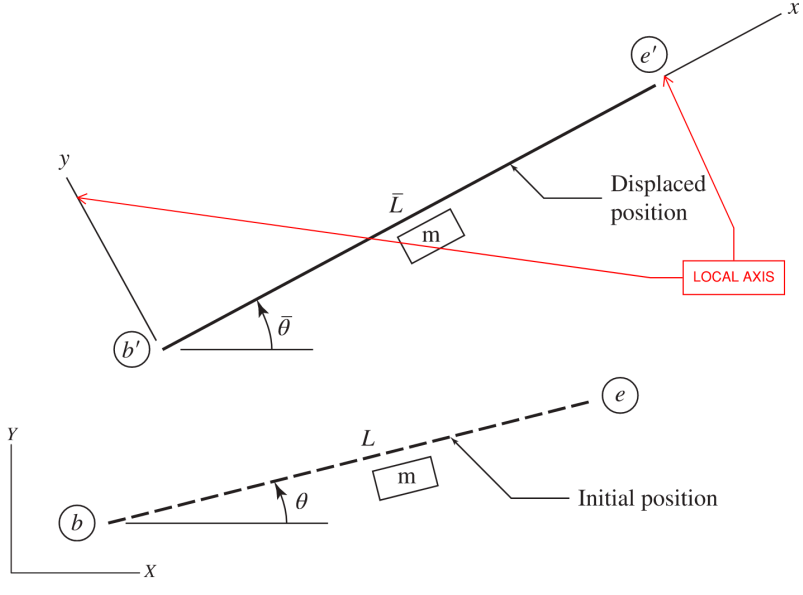
So the local x-axis is always directed along the member centroidal axis in its **current deformed state**.

This moving local system is often called a **corotational** or **Eulerian** local coordinate system.

Corotational / Eulerian Local Coordinate System

For a plane truss member:

- the origin of the local system is always at the **beginning node** of the member in its deformed state
- the local x-axis is directed along the **deformed member axis**
- the positive local y-axis is chosen so the coordinate system remains **right-handed**



Why This Coordinate Choice Helps

For a truss member, the only true member deformation is **axial stretching or shortening**.

By aligning the local axis with the **current deformed member direction**, we can write the member constitutive law in a very simple form.

So even though the structure moves and rotates in space, the member still behaves locally like an axial bar.

Part 2 — Member Force-Displacement Relations in the Local System

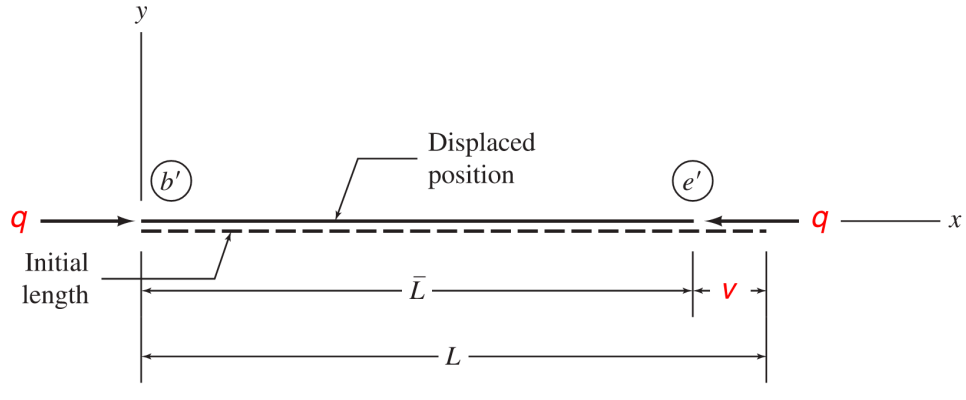
Axial Deformation in the Local System

Let a plane truss member have:

- undeformed length L
- deformed length $\bar{L}$

Then the member axial deformation is defined as

$$v = L - \bar{L} \tag{1}$$



With this sign convention:

- $v > 0$ corresponds to **shortening**
- $v < 0$ corresponds to **elongation**

Likewise, the member axial force q is taken as:

- positive in **compression**
- negative in **tension**

Stress, Strain, and Constitutive Law

For the truss member,

$$\sigma = \frac{q}{A} \qquad \text{and} \qquad \varepsilon = \frac{v}{L} \tag{2}$$

For a linearly elastic material,

$$\sigma = E\varepsilon \tag{3}$$

Substituting Eq. (2) into Eq. (3) gives the member force-displacement relation in the local system:

$$q = \left(\frac{EA}{L}\right) v \tag{4}$$

So locally, the member still behaves like an axial spring.

In [83]:

```
import numpy as np

# Example material and member properties
E = 70e3       # MPa = N/mm^2
A = 645.2      # mm^2
L = 3000.0     # mm

# Local axial deformations (positive = shortening with this sign convention)
u_vals = np.array([10.0, 0.0, -10.0])  # mm
q_vals = (E * A / L) * u_vals

for u, q in zip(u_vals, q_vals):
    state = 'compression' if q > 0 else ('tension' if q < 0 else 'zero force')
    print(f'v = {u:6.1f} mm  ->  q = {q:10.1f} N   ({state})')
```

```
v =   10.0 mm  ->  q =   150546.7 N   (compression)
v =    0.0 mm  ->  q =        0.0 N   (zero force)
v =  -10.0 mm  ->  q =  -150546.7 N   (tension)
```

Important Observation

Eq. (4) has the same form as the familiar linear axial bar relation.

The key point is that the **material law itself is still linear**.

The geometric nonlinearity enters elsewhere:

- through the changing member length $\bar{L}$
- through the changing member orientation
- and through the transformation from local to global quantities

Note that because of the modified definition of the local coordinate system, only one degree of freedom (that is the member axial deformation) is now needed to completely specify the displaced position of the member.

Part 3 — Local to Global Transformations

Initial and Deformed Member Length

Suppose a member connects nodes b and e.

Using the **initial** nodal coordinates,

$$L = \sqrt{(X_e - X_b)^2 + (Y_e - Y_b)^2} \tag{5}$$

If the global nodal displacements are

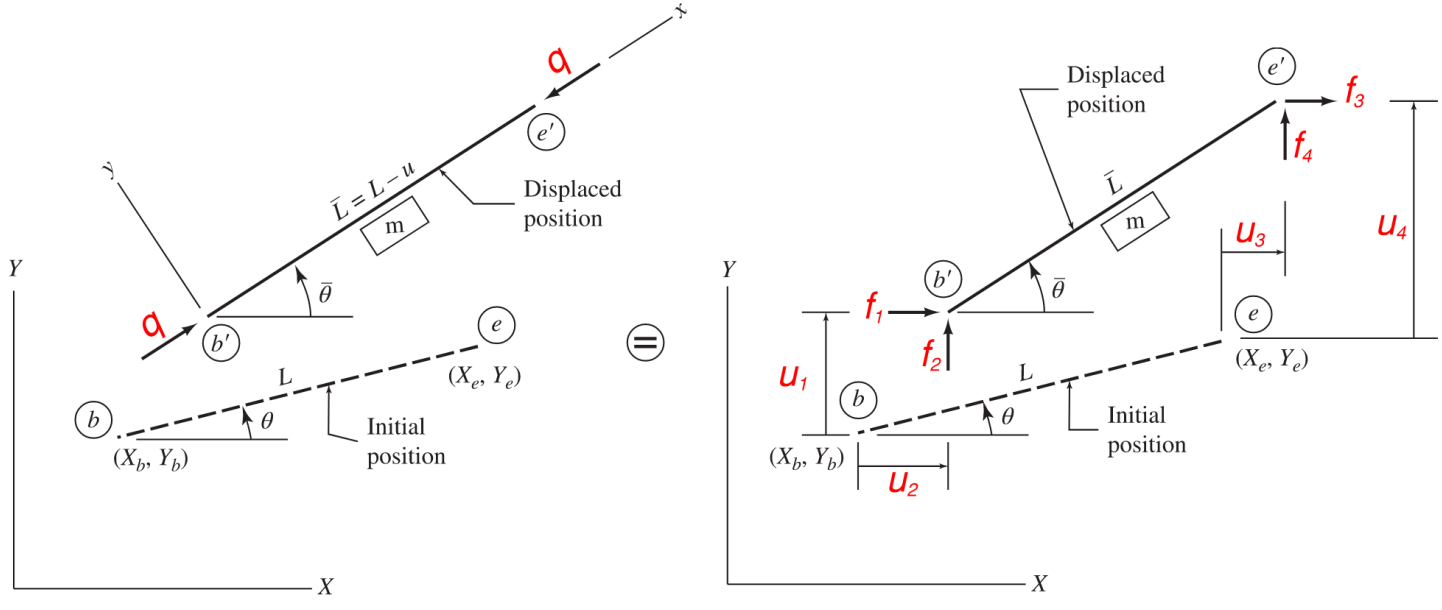
$$\mathbf{u} = [u_1 \; u_2 \; u_3 \; u_4]^T$$

with:

- u_1, u_2 = X- and Y-displacements (global) at node b
- u_3, u_4 = X- and Y-displacements (global) at node e

then the **deformed** member length is

$$\bar{L} = \sqrt{\left[(X_e + u_3) - (X_b + u_1)\right]^2 + \left[(Y_e + u_4) - (Y_b + u_2)\right]^2} \tag{6}$$



Current Direction Cosines

In the **deformed configuration**, the member orientation is described by the current angle $\bar{\theta}$.

The corresponding direction cosines are

$$c_x = \cos\bar{\theta} = \frac{(X_e + u_3) - (X_b + u_1)}{\bar{L}} \tag{7}$$

$$c_y = \sin\bar{\theta} = \frac{(Y_e + u_4) - (Y_b + u_2)}{\bar{L}} \tag{8}$$

Here, $\bar{\theta}$ is measured **counterclockwise** from the positive global X-axis to the positive local x-axis of the member in its **deformed** position.

Unlike in linear analysis, these quantities are **not constant**. Because they depend on the current nodal displacements, they change continuously as the structure deforms.

In [84]:
```
# Example: compute deformed length and direction cosines from nodal displaceme

Xb, Yb = 0.0, 0.0
Xe, Ye = 4000.0, 3000.0

u1, u2 = 20.0, 10.0
u3, u4 = 80.0, -40.0

L0 = np.sqrt((Xe - Xb)**2 + (Ye - Yb)**2)
Lbar = np.sqrt(((Xe + u3) - (Xb + u1))**2 + ((Ye + u4) - (Yb + u2))**2)
cx = ((Xe + u3) - (Xb + u1)) / Lbar
cy = ((Ye + u4) - (Yb + u2)) / Lbar
u_local = L0 - Lbar

print(f'Initial length L      = {L0:.3f} mm')
print(f'Deformed length Lbar  = {Lbar:.3f} mm')
print(f'c_x = {cx:.6f}')
print(f'c_y = {cy:.6f}')
print(f'Local axial deformation u = {u_local:.3f} mm')
```

```
Initial length L      = 5000.000 mm
Deformed length Lbar  = 5018.575 mm
c_x = 0.808995
c_y = 0.587816
Local axial deformation u = -18.575 mm
```

From Local Axial Force to Global End Forces

At node b, the global end-force components are the components of the local axial force q along the global axes:

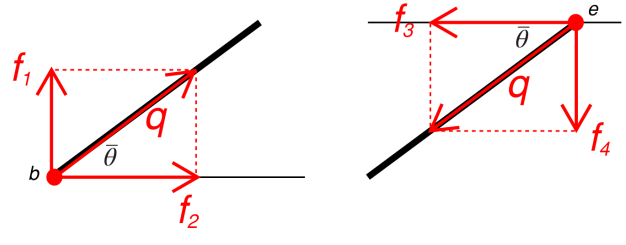
$$f_1 = c_x q \tag{9a}$$

$$f_2 = c_y q \tag{9b}$$

At node e, the equal-and-opposite components are

$$f_3 = -c_x q \tag{9c}$$

$$f_4 = -c_y q \tag{9d}$$



Transformation Form

So the global member end-force vector is

$$\mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} = \begin{bmatrix} c_x \\ c_y \\ -c_x \\ -c_y \end{bmatrix} q \tag{10}$$

We can write Eq. (10) more compactly as

$$\mathbf{f} = \mathbf{T}^T q \tag{11}$$

where the 1×4 transformation matrix is

$$\mathbf{T} = \begin{bmatrix} c_x & c_y & -c_x & -c_y \end{bmatrix} \tag{12}$$

In [85]:
```
# Continue from the previous example

q = (E * A / L0) * u_local
T = np.array([[cx, cy, -cx, -cy]])
f = T.T * q

print(f'Local axial force q = {q/1000:.3f} kN')
print('\nTransformation matrix T =')
print(T)

print('\nGlobal end-force vector f =')
for i, val in enumerate(f.flatten(), start=1):
    print(f'f_{i} = {val/1000:.1f} kN')
```

```
Local axial force q = -167.789 kN

Transformation matrix T =
[[ 0.80899451  0.5878162  -0.80899451 -0.5878162 ]]

Global end-force vector f =
f_1 = -135.7 kN
f_2 = -98.6 kN
f_3 = 135.7 kN
f_4 = 98.6 kN
```

Conceptual Summary of the Member Relations

To evaluate the member global end-force vector $\mathbf{f}$ from known global nodal displacements $\mathbf{u}$, the steps are:

1. compute the initial length L using Eq. (5)
2. compute the deformed length $\bar{L}$ using Eq. (6)
3. compute the current direction cosines c_x, c_y using Eqs. (7) and (8)
4. compute the local axial deformation $u = L - \bar{L}$ using Eq. (1)
5. compute the local axial force q from Eq. (4)
6. transform q to the global end-force vector $\mathbf{f}$ using Eq. (11)

Why the Global Member Relations Are Nonlinear

Even though the local constitutive law

$$q = \left(\frac{EA}{L}\right) u$$

is linear, the **global** member relations are geometrically nonlinear because:

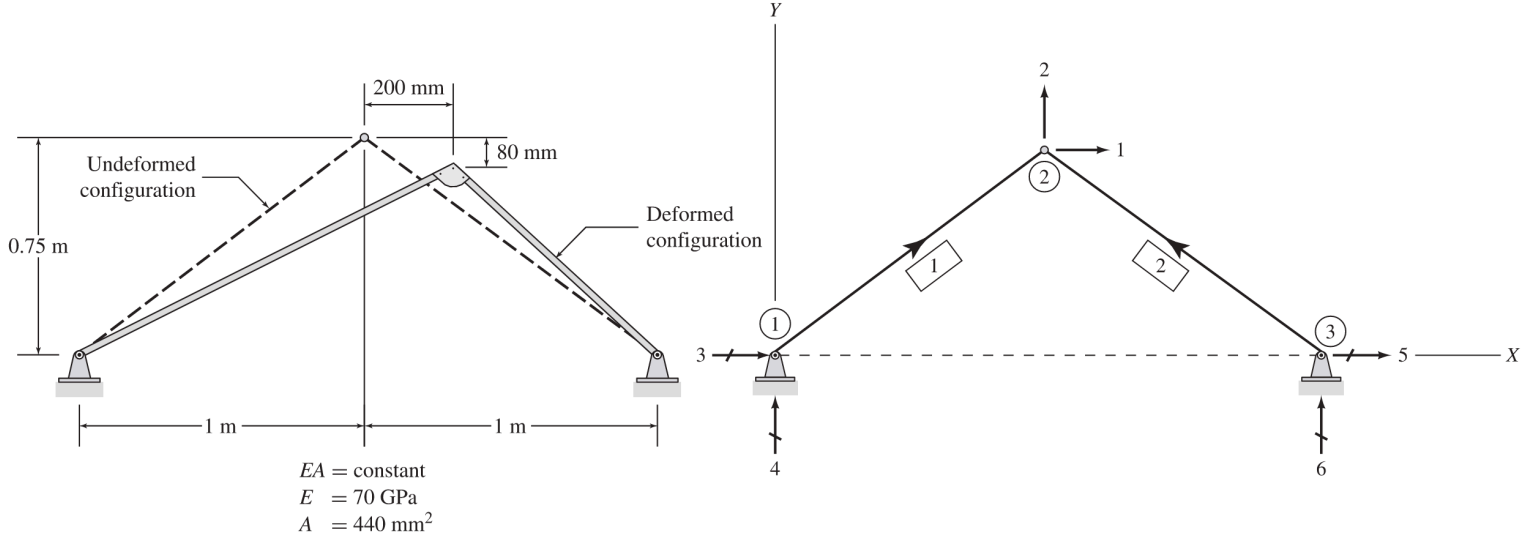
- $u = L - \bar{L}$ depends nonlinearly on the global nodal displacements
- c_x and c_y depend on the current deformed orientation
- the transformation matrix $\mathbf{T}$ therefore changes during deformation

So the member force vector $\mathbf{f}$ is ultimately a nonlinear function of the global displacement vector $\mathbf{u}$.

Part 4 — In-Class Example (Given Displacements)

Example Problem

By using geometrically nonlinear analysis, determine the joint loads P that cause the two-member truss to deform into the configuration shown



In [86]:

```
import numpy as np

def truss_deformed_geometry(start_point, end_point, deformations):
    Xb, Yb = start_point
    Xe, Ye = end_point
    u1, u2, u3, u4 = deformations

    L0 = np.sqrt((Xe - Xb)**2 + (Ye - Yb)**2)
    Lbar = np.sqrt(((Xe + u3) - (Xb + u1))**2 + ((Ye + u4) - (Yb + u2))**2)
    cx = ((Xe + u3) - (Xb + u1)) / Lbar
    cy = ((Ye + u4) - (Yb + u2)) / Lbar
    u_local = L0 - Lbar

    return {
        "L0": L0,
        "Lbar": Lbar,
        "cx": cx,
        "cy": cy,
        "u_local": u_local,
    }
```

In [87]:

```
# Calculate member geometric parameters
m1 = truss_deformed_geometry((0, 0), (1000, 750), (0, 0, 200, -80))
m2 = truss_deformed_geometry((0, 0), (-1000, 750), (0, 0, 200, -80))

E = 70e3  # MPa = N/mm^2
A = 440.0   # mm^2

# calculate local axial forces
q1 = (E * A / m1["L0"]) * m1["u_local"]
q2 = (E * A / m1["L0"]) * m2["u_local"]

print(f'Member 1: q = {q1/1000:.3f} kN')
print(f'Member 2: q = {q2/1000:.3f} kN')
```

```
Member 1: q = -3064.540 kN
Member 2: q = 5088.069 kN
```

In [88]:

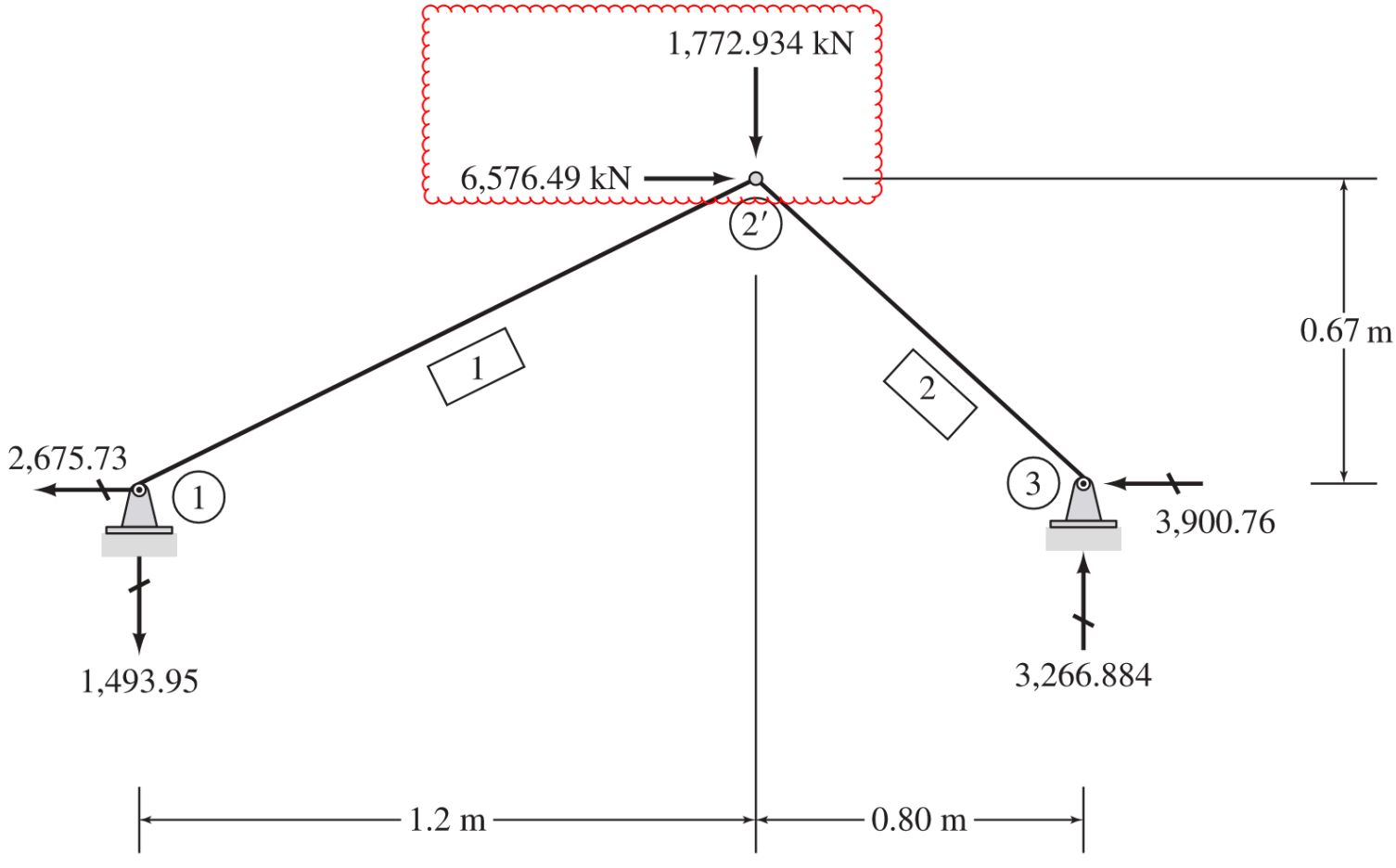
```
# Tranform to global forces
T1 = np.array([[m1["cx"], m1["cy"], -m1["cx"], -m1["cy"]]])
T2 = np.array([[m2["cx"], m2["cy"], -m2["cx"], -m2["cy"]]])
f1 = T1.T * q1
f2 = T2.T * q2

print("Global end forces (kN)")
print(
    f"m1: "
    f"f1 = {f1[0,0]/1000:.1f}   "
    f"f2 = {f1[1,0]/1000:.1f}   "
    f"f3 = {f1[2,0]/1000:.1f}   "
    f"f4 = {f1[3,0]/1000:.1f}"
)

print(
    f"m2: "
    f"f1 = {f2[0,0]/1000:.1f}   "
    f"f2 = {f2[1,0]/1000:.1f}   "
    f"f3 = {f2[2,0]/1000:.1f}   "
    f"f4 = {f2[3,0]/1000:.1f}"
)
```

```
Global end forces (kN)
m1: f1 = -2675.7   f2 = -1493.9   f3 = 2675.7   f4 = 1493.9
m2: f1 = -3900.8   f2 = 3266.9   f3 = 3900.8   f4 = -3266.9
```

Add together f3 and f4 for both members to get X and Y forces at node 2



Wrap-Up

In this lecture, we:

- introduced the **corotational / Eulerian local coordinate system** for a plane truss member
- defined the member axial deformation and local constitutive law
- expressed the deformed member length and current direction cosines in terms of global nodal displacements
- transformed the local axial force q into the global member end-force vector $\mathbf{f}$
- saw that the member law is locally linear, but globally **geometrically nonlinear**

Next: we will use these member relations to begin forming the **geometrically nonlinear load-displacement relations** for entire truss structures.